Bus Timetable and Ticketing System

Report

Team name: Runtime Terrors

Team members:

Florin Stefan Grosu (M00937809) – Team Leader

Ugur Ersoy (M00963545) – Developer

Akram Mohamed (M00983353) – Developer

Luis Oliveira (M00946476) – Tester

Bogdan Marian Nicolescu (M01034693) - Secretary

Repository:

RuntimeTerrors29/Bus-Timetable-Coursework

Introduction

This project is a Bus Timetable and Ticketing System built in C# (.NET 8), developed by a team of 5 members over 3 sprints across 15 days. The program runs as a console app and allows passengers to search for bus stops, view the timetable and book or cancel tickets.

There is also a separate admin menu that allows staff to update the capacity on a route. Everything is stored in a SQLite database and the user picks the database file path when the program starts, therefore nothing is hardcoded.

The main challenge of the project was that we were not allowed to use any STL data structures such as List <> or Dictionary <>. We had to build the data structures ourselves from scratch. That ended up shaping most of the decisions we made. We built 3 structures, a hash table with separate chaining for looking up bus stops quickly, a sorted linked list to keep the timetable in departure-time order, and a plain linked list for tickets and passengers.

Furthermore, this report will cover the design and why we picked each structure and pseudo code for the key operations, test cases we wrote and our testing approach and the conclusion where we summarise what has been accomplished, reflect on the problems we ran into and discuss what we would do differently next time. References are attached at the end in Harvard style.

Design

We required 3 data structures for this project, and the choice of each came down to the operations needed to be performed on the data and we could not use anything from the standard library.

Bus Stop Hash Table

Bus stops get looked up frequently by ID when booking, and by partial name when a passenger is searching. A linked list would mean scanning the whole thing every time, which is O(n). A hash table gets us to O(1) average case for ID lookups, which is a lot more efficient. We decided to use separate chaining for collisions because it handles them cleanly in the absolute worst case where every stop lands in the same bucket, you are back to O(n), but with 64 buckets and when all the IDs are spread into different buckets, that basically never happens (Cormen et al., 2022). Each bucket is a small ChainNode linked list we wrote ourselves, so there are no standard collections anywhere in the structure.

Timetable List

Schedules need to come out in departure-time order and we need to be able to filter by a time range. A sorted linked list works well for this insertion as it keeps the sort order at O(n) worst case, but we only insert a startup when loading from the database so that cost is only paid once. The real benefit is in GetBetween because the list is already sorted and we only need to scan from head until we get a match. Once we have a match we can break from the loop. Most queries only visit a part of the list rather than going through the whole thing. (Goodrich, Tamassia and Goldwasser, 2014). We considered using binary sort or binary search tree, but we came to the conclusion that it will not be the best fit for a linked-list because binary search works on sorted lists and halves the list each time, therefore not suitable for what we wanted to achieve.

Ticket List & Passenger List

Tickets and passengers only need to be looked up by ID or filtered by passenger, no sorting needed, and we always add new entries at the head so that is O(1). A plain unsorted linked list is the right fit because when there is a query on either of the list, rather than returning the index or the array they are in, we get the information and then create a fixed sized array and pass that array. It doubles the process which makes it less efficient but it works as expected.

Algorithm Analysis

BusStopHashTable.Add

```
FUNCTION Add(stop):
  index ← |stop.StopID MOD tableSize|
  current ← buckets[index]
  WHILE current ≠ null:
    IF current.Data.StopID = stop.StopID THEN
      current.Data ← stop          // update in place
      RETURN
    END IF
    current ← current.Next
  END WHILE
  newNode ← new ChainNode(stop)
  newNode.Next ← buckets[index]    // prepend to chain
  buckets[index] ← newNode
  count ← count + 1
END FUNCTION
```

Time complexity O(1) average. If there is already a stop with the same stop ID it gets updated in place and count does not change. Math.Abas stops negative IDs from causing an index out of range error which was a bug in earlier development.

BusStopHashTable.SearchByName(partial match)

```
FUNCTION SearchByName(partial):
  matchCount ← 0
  FOR i FROM 0 TO tableSize - 1:
    current ← buckets[i]
    WHILE current ≠ null:
      IF current.Data.StopName CONTAINS partial THEN
        matchCount ← matchCount + 1
      END IF
      current ← current.Next
    END WHILE
  END FOR
  results ← new array[matchCount]
  idx ← 0
  FOR i FROM 0 TO tableSize - 1:       // second pass
    current ← buckets[i]
    WHILE current ≠ null:
      IF current.Data.StopName CONTAINS partial THEN
        results[idx] ← current.Data
        idx ← idx + 1
      END IF
      current ← current.Next
    END WHILE
  END FOR
  RETURN results
END FUNCTION
```

Time complexity: O(n) you have to scan the whole table because partial name matching cannot use the hash function. The 2 pass approach is needed because we cannot use List<T> to collect results dynamically.

TimetableList.InsertSorted

```
FUNCTION InsertSorted(schedule):
  newNode ← new Node(schedule)
  IF head = null OR schedule.DepartureTime ≤ head.Data.DepartureTime THEN
    newNode.Next ← head
    head ← newNode
    count ← count + 1
    RETURN
  END IF
  current ← head
  WHILE current.Next ≠ null AND
        current.Next.Data.DepartureTime ≤ schedule.DepartureTime:
    current ← current.Next
  END WHILE
  newNode.Next ← current.Next
  current.Next ← newNode
  count ← count + 1
END FUNCTION
```

Time complexity: O(n) worst case, when the new departure is the latest in the list. In practice this is fine because all inserts happen at startup when loading from the database, therefore once the data is loaded the list is not written to again.

TimetableListGetBetween(time-range query)

```
FUNCTION GetBetween(from, to):
  matchCount ← 0
  current ← head
  WHILE current ≠ null:
    dep ← current.Data.DepartureTime
    IF dep ≥ from AND dep ≤ to THEN
      matchCount ← matchCount + 1
    END IF
    IF dep > to THEN BREAK          // early exit — list is sorted
    current ← current.Next
  END WHILE
  results ← new array[matchCount]
  // second pass with same early-exit logic
  ...
  RETURN results
END FUNCTION
```

Time complexity: O(k) where k is the number of nodes visited before DepartureTime goes past the upper bound. Because the list is sorted we can break early, therefore most queries only touch a small part of the list rather than the whole thing.

TimetableList.Update

```
FUNCTION Update(updated):
  current ← head
  WHILE current ≠ null:
    IF current.Data.ScheduleID = updated.ScheduleID THEN
      current.Data ← updated
      RETURN true
    END IF
    current ← current.Next
  END WHILE
  RETURN false
END FUNCTION
```

Time complexity: O(n) This is used by the admin UpdateCapacity option after DatabaseManager.UpdateSchedule writes the new capacity to the database. Update replaces schedule object in the linked list so in the in-memory timetable reflects the change straight-away.

TicketList.Cancel

```
FUNCTION Cancel (ticketId) :
  current  ←  head
  WHILE current ≠ null:
    IF current.Data.TicketID  =  ticketId THEN
       IF current.Data.Status  =  "Cancelled"  THEN
                RETURN false              //idempotency guard
        END IF
      current.Data.Status  ←  "Cancelled"
      RETURN true
    END IF
    current  ←  current.Next
  END WHILE
 RETURN false              //not found
END FUNCTION
```

Time complexity: O(n) The check for an already-cancelled ticket was added as a bug fix in Sprint 2, the original just set the status and returned true regardless. MenuController checks the return value before calling DatabaseManager, CancelBooking so it will not hit the database if the cancel was not actually needed.

System Architecture

Program.cs is the entry point. It asks the user for a database path when it starts (defaulting to bus.db if they just press Enter), then calls DatabaseManager to create the schema if it does not already exist, seed the sample data on first launch, and load all the records into the data structures we have. After that it hands control to MenuController, which holds references (as the data structures are in-memory) to all 4 structures.

The SQLite schema has six tables: BusStops, BusRoutes, RouteStops, Schedules, Passengers, and Tickets. The sample data file seeds 12 London – area stops, 4 routes, route-stop links with distances and travel times, a set of schedules spread across the day, 5 passengers and 5 tickets with 1 already cancelled.

Database Manager has separate methods for loading (LoadStops, LoadSchedules, LoadTickets, LoadPassengers) and writing (AddPassenger, AddBooking, CancelBooking, UpdateSchedule). LoadSchedules joins with BusRoutes, therefore each schedule object carries its RouteName for display. AddBooking runs a Follow-up UPDATE on Schedules.SeatsBooked straight after inserting the ticket row so the database seat count stays in sync with the in-memory timetable.

Testing

We used MSTest to write unit tests for both the model classes and the custom data structure(Microsoft, 2024a). Tests were written, once we finished building the project. A few bug fixes were done while building it and then in order to make sure everything was properly implemented, we started working on test cases.

Tests are split into files by class: BusStopTests, BusRouteTests, ScheduleTests, PassengerTests, TicketTests, and DataStructuresTests which covers the hash table and both linked list together. DataStructuresTests was the most important file since the data structures are the core of what was being assessed.

Test Cases

Test ID	Description	Expected result	Outcome	Class
1	Add a bus stop to BusStopHashTable and retrieve by ID	GetById returns correct BusStop object	Pass	BusStopHashTableTest
2	Add duplicate StopID to hash table (regression TC)	Existing entry overwritten, count unchanged	Pass	BusStopHashTableTest
3	Hash functions receives negative stopID (regression TC)	Math.Abs prevents index error / stop stored correctly	Pass	BusStopHashTableTest
4	SearchByName with partial string match	All matching stops returned in array	Pass	BusStopHashTableTest
5	Remove stop from middle of collision chain	Stop removed, count decremented, others unaffected	Pass	BusStopHashTableTest
6	GetById on empty hash table (regression TC)	Returns null	Pass	`BusStopHashTableTests`
7	InsertSorted maintains departure-time order	Schedules retrieved in ascending time order	Pass	TimetableListTest
8	GetBetween exits early beyond upper bound	Only schedules in range returned, traversal stops	Pass	TimetableListTest
9	Remove only element from TimetableList	Head set to null, count becomes 0	Pass	TimetableListTest
10	TimetableList.Update modifies schedule in place	Updated schedule returned by GetById	Pass	TimetableListTest

11	Cancel already-cancelled ticket (regression TC)	Returns false, status unchanged	Pass	TicketListTest
12	GetByPassenger with no matching tickets	Empty array returned	Pass	TicketListTest
13	AvailableSeats = Cpacity-SeatsBooked(regression TC)	Correct seat count returned after booking	Pass	ScheduleTest

Conclusion

In conclusion, we delivered a working C# console application for Bus Timetable and Ticketing System. Passengers can search for stops by partial name, view the full timetable or filter by toute, book a ticket by entering their name, email and schedule ID, cancel a ticket and check their booking history by passenger ID. There is also an admin menu where staff can update the capacity on a schedule, with a check that prevents it being set below, however many seats are already booked.

We built 4 custom data structures without using any standard library collections, the hash table with separate chaining, the sorted linked list for the timetable, and a plain linked list for tickets and passengers. The schema has 6 tables and the sample data file seeds 12 London-based stops, 4 routes with stop links including distances and travel times, schedules across the day and 5 passengers along with 5 tickets (1 already cancelled, so the cancel flow can be tested from the first launch).

We wrote 13 MSTest cases covering the data structures and model classes, with regression tests for all 9 bugs we found and fixed across the project. (Microsoft, 2024b).

Limitations & Reflection

The biggest problem we ran into was keeping the database and the in-memory structures in sync. Three separate bugs came from this: Add Booking did not run the follow-up UPDATE on Schedules.SeatsBooked after inserting a ticket row, CancelTicket did not decrement SeatsBooked in the TimetableList after cancelling, and the AvailableSeats property in Schedule was doing addition instead of subtraction. All 3 have been fixed, but the fact that they all appeared in the same area shows a design problem with MenuController and DatabaseManager as they were developed in parallel by 2 different members without a clear enough agreement about which layer was responsible for what. A service layer sitting between the menu and the database with a defined API for each booking action would have made the responsibilities much cleaner and probably prevented at least some of these bugs.

Another issue was that LoadTickets was initially missing the join with BusRoutes, therefore RouteName came back blank on every loaded ticket. This was caught and fixed, but it happened because the team had not fully agreed on what database query needed to return before writing them.

The hash table size is fixed at 64 buckets and never resizes. If the number of stops got much larger the load factor would go up an collision chains would get longer, pushing average-case lookup towards O(n).

Future approach

The main thing we would do differently is decide the interface between database layer and the in-memory structures before anyone writes any code. Writing out in pseudo code which operations touch both layers and in what order would have caught the SeatsBooked synchronisation bugs at the design stage rather than during testing. It would also have removed the ambiguity about which team member was responsible for each part of a combined operation like booking or cancelling.

We would also write tests before implementing each method rather than adding them after. Several bugs were only found because someone noticed unexpected output while running the program manually. A test written first would have caught the same thing immediately.

Finally, we would spend more time on the Sprint 1 design document. We started writing code in parallel before everyone had a complete shared understanding of how all components fit together. A good design review at the end of Sprint 1, even just walking through the component interactions as a team, would have saved time in Sprint 2, when we kept running into integration issues between MenuController and DatabaseManager.

References

● Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C. (2022) Introduction to Algorithms. 4th edn. Cambridge, MA: MIT Press.

● Goodrich, M.T., Tamassia, R. and Goldwasser, M.H. (2014) Data Structures and Algorithms in Java. 6th edn. Hoboken, NJ: Wiley.

● Microsoft (2024a) MSTest overview. Available at: https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-with-mstest (Accessed: 20 March 2026).

● Microsoft (2024b) Microsoft.Data.Sqlite overview. Available at: https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/ (Accessed: 14 March 2026).

● Sedgewick, R. and Wayne, K. (2011) Algorithms. 4th edn. Upper Saddle River, NJ: Addison-Wesley.